Práctica 4 Grupo B9

Francisco Javier Martín Lunas Escobar

Pedro Jesús Cros Pérez

UNIVERSIDAD DE JAÉN

Índice

Auditoría de casos de estudio **2**
Caso 1 2
Captura “Antes” 2
Promt 2
Test 3
Captura “Después” 3
Caso 2 4
Captura “Antes” 4
Promt 4
Test 5
Captura “Después” 6
Caso 3 7
Captura “Antes” 7
Promt 7
Test 8
Captura “Después” 9
Evolución de la cobertura **10**
Lanzamiento a producción **13**

Francisco Javier Martín Lunas Escobar
Pedro Jesús Cros Pérez

Auditoría de casos de estudio

Caso 1

Captura “Antes”

```
Laboratorio.h   ListaEnlazada.h   core_logic.cpp   Laboratorio.cpp   ListaEnlaza
src > core_logic.cpp > safe_read(unsigned int &)
17  static void mostrarLista(const ListaEnlazada<int> &lista) {
20      while (!it.isEnd()) {
23      }
24      std::cout << "]" << std::endl;
25  }
26
27  /**
28   * @brief Imprime por consola un separador visual con un título centrado.
29   * @param titulo Texto a mostrar como título del separador.
30   */
31  static void separador(const std::string &titulo) {
32      std::cout << "==========================================================================
33      std::cout << "------------------------------------------- " << titulo << " ---------
34      std::cout << "==========================================================================
35  }
36
37  bool safe_read(unsigned int& value) {
38      if (std::cin >> value) return true;
39      if (std::cin.eof()) return false;
40      std::cin.clear();
41      std::cin.ignore(10000, '\n');
42      return true;
43  }
44
45  /**
46   * @brief Realiza las inserciones de la Prueba I sobre la lista de enteros.
47   * @param lista Lista enlazada sobre la que operar.
48   */
49  static void prueba_lista_inserciones(ListaEnlazada<int>& lista) {
50      for (int i = 101; i <= 200; ++i) lista.insertAtEnd(i);
```

Promt

Actúa como un experto en C++ y Google Test. Estoy realizando un análisis de cobertura de caja blanca. Tengo una función en la que tres ramas NO se están ejecutando con mis tests actuales.

CONTEXTO DE MIS TESTS: Te adjunto mi archivo de pruebas actual (test_main.cpp) para que conozcas la estructura y los Fixtures que utilizo.

CÓDIGO DE LA FUNCIÓN COMPLETA:

```
bool safe_read(unsigned int& value) {
    if (std::cin >> value) return true;
```

```
    if (std::cin.eof()) return false;
    std::cin.clear();
    std::cin.ignore(10000, '\n');
    return true;
}
```

BLOQUES O CONDICIONES NO CUBIERTAS (marcadas en rojo):

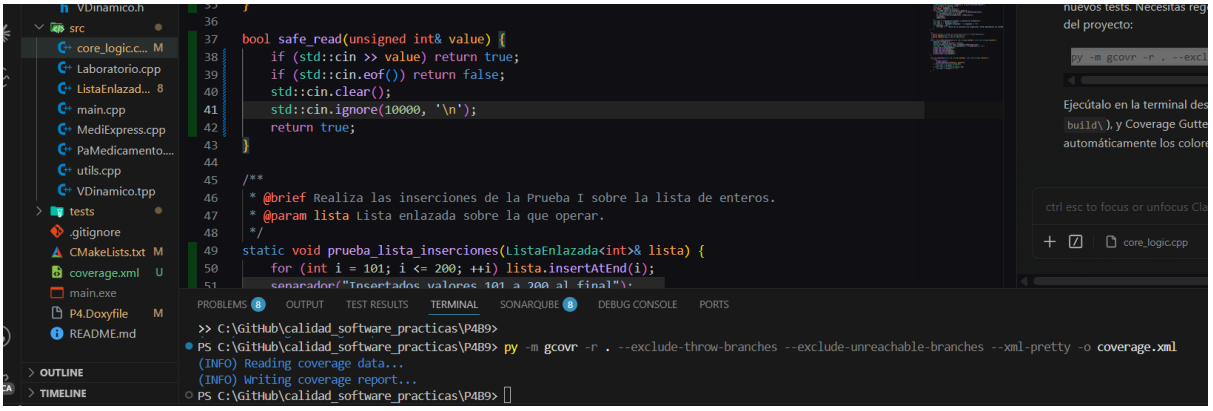
Línea 38: if (std::cin >> value) return true;
Línea 39: if (std::cin.eof()) return false;
Líneas 40-42: std::cin.clear(); std::cin.ignore(10000, '\n'); return true;
REQUISITOS OBLIGATORIOS:

Genera ÚNICAMENTE tres bloques TEST que fuercen la ejecución exacta de cada una de esas líneas en rojo.
NO generes includes (#include).
NO generes la función main().
NO reescribas el archivo completo. Solo quiero los nuevos bloques listos para copiar y pegar.
MUY IMPORTANTE: Analiza el test_main.cpp adjunto. La función safe_read() lee de std::cin, por lo que DEBES redirigir std::cin a un std::istringstream antes de llamarla y restaurarlo después. NO uses el Fixture SimulationTest para estos tests, ya que no redirigen std::cin; usa TEST simple.

Test

a

Captura “Después”



Francisco Javier Martín Lunas Escobar
Pedro Jesús Cros Pérez

Caso 2

Captura “Antes”

```
142  */
143  static void prueba_labs_aceites(const MediExpress& me) {
144      separador("Labs sumistran Aceites");
145      VDinamico<PaMedicamento*> aceites = me.buscarCompuesto("aceite");
146      ListaEnlazada<const Laboratorio*> labsUnicos;
147      for (unsigned int i = 0; i < aceites.len(); ++i) {
148          const Laboratorio* lab = aceites[i]->getServidoPor();
149          if (!lab) continue;
150          bool yaEsta = false;
151          for (auto itL = labsUnicos.iterator(); !itL.isEnd(); itL.next())
152              if (itL.data() == lab) { yaEsta = true; break; }
153          }
154          if (!yaEsta) labsUnicos.insertAtEnd(lab);
155      }
156      std::cout << "\nLaboratorios que suministran 'ACEITES': " << labsUni
157      int idx = 1;
158      for (auto itL = labsUnicos.iterator(); !itL.isEnd(); itL.next()) {
159          std::cout << idx << ". " << *itL.data() << "\n";
160          ++idx;
161      }
162      if (labsUnicos.size() == 0) std::cout << "(Ninguno)\n";
163  }
```

Promt

Actúa como un experto en C++ y Google Test. Estoy realizando un análisis de cobertura de caja blanca. Tengo una función en la que una rama específica NO se está ejecutando con mis tests actuales.

CONTEXTO DE MIS TESTS: Te adjunto mi archivo de pruebas actual (test_main.cpp) para que conozcas la estructura y los Fixtures que utilizo.

CÓDIGO DE LA FUNCIÓN COMPLETA:

```
static void prueba_labs_aceites(const MediExpress& me) {
    separador("Labs sumistran Aceites");
    VDinamico<PaMedicamento*> aceites = me.buscarCompuesto("aceite");
    ListaEnlazada<const Laboratorio*> labsUnicos;
    for (unsigned int i = 0; i < aceites.len(); ++i) {
        const Laboratorio* lab = aceites[i]->getServidoPor();
        if (!lab) continue;
        bool yaEsta = false;
        for (auto itL = labsUnicos.iterator(); !itL.isEnd(); itL.next()) {
            if (itL.data() == lab) { yaEsta = true; break; }
```

Francisco Javier Martín Lunas Escobar
Pedro Jesús Cros Pérez

```
        }
        if (!yaEsta) labsUnicos.insertAtEnd(lab);
    }
    ...
}
```

BLOQUE NO CUBIERTO (marcado en rojo): if (!lab) continue;

REQUISITOS OBLIGATORIOS:

Genera ÚNICAMENTE un bloque TEST_F que fuerce que aceites[i]->getServidoPor() devuelva nullptr, ejecutando el continue.
La función se invoca a través de run_simulation(), por lo que debes usar el Fixture SimulationTest y controlar los CSVs mediante writeFile().
Para que un medicamento con "aceite" en el nombre quede sin laboratorio, crea 5 medicamentos y solo 2 laboratorios: autoLinkMedications() asignará lab a los 4 primeros (2 por lab) y el 5.º quedará sin lab.
NO generes #include, ni main(), ni reescribas el archivo. Solo el bloque listo para pegar.

El test anterior cubre el continue de la línea 149, pero la línea 152 (if (itL.data() == lab) { yaEsta = true; break; }) sigue en rojo. Esa rama se ejecuta cuando dos medicamentos con "aceite" comparten el mismo laboratorio y el segundo intenta insertarlo de nuevo en labsUnicos.

Genera un TEST_F adicional con 2 medicamentos que contengan "aceite" y UN SOLO laboratorio, de modo que autoLinkMedications() asigne ese mismo lab a ambos. Al procesar el segundo medicamento, el bucle interior encontrará el lab ya existente y ejecutará yaEsta = true; break.

Test

```
TEST_F(SimulationTest, PruebaLabsAceites_SkipsNullLab) {
    writeFile(TEST_MEDS,
        "1;MED001;Paracetamol;\n"
        "2;MED002;Ibuprofeno;\n"
        "3;MED003;Amoxicilina;\n"
        "4;MED004;Omeprazol;\n"
        "5;MED005;Aceite Esencial;\n");
    writeFile(TEST_LABS,
        "1;FarmaGen S.L.;Calle Mayor 1;18001;Granada\n"
        "2;HealthMed S.A.;Calle Alcala 10;28001;Madrid\n");

    run_simulation(TEST_MEDS, TEST_LABS);

    const std::string output = cout_buffer.str();
    EXPECT_NE(output.find("(Ninguno)"), std::string::npos)
        << "El med con aceite sin lab debe ser omitido, resultando en 0 laboratorios.";
}
```

Francisco Javier Martín Lunas Escobar
Pedro Jesús Cros Pérez

```
TEST_F(SimulationTest, PruebaLabsAceites_DetectsDuplicateLab) {
    writeFile(TEST_MEDS,
        "1;MED001;Aceite de Almendras;\n"
        "2;MED002;Aceite de Oliva;\n");
    writeFile(TEST_LABS,
        "1;FarmaGen S.L.;Calle Mayor 1;18001;Granada\n");

    run_simulation(TEST_MEDS, TEST_LABS);

    const std::string output = cout_buffer.str();
    EXPECT_NE(output.find("Laboratorios que suministran 'ACEITES': 1"), std::string::npos)
        << "Ambos meds aceite tienen el mismo lab: debe haber 1 unico en labsUnicos.";
}
```

Captura “Después”

```
140  * @brief Muestra los laboratorios únicos que suministran medicamentos c
141  * @param me Sistema MediExpress sobre el que consultar.
142  */
143 static void prueba_labs_aceites(const MediExpress& me) {
144     separador("Labs sumistran Aceites");
145     VDinamico<PaMedicamento*> aceites = me.buscarCompuesto("aceite");
146     ListaEnlazada<const Laboratorio*> labsUnicos;
147     for (unsigned int i = 0; i < aceites.len(); ++i) {
148         const Laboratorio* lab = aceites[i]->getServidoPor();
149         if (!lab) continue;
150         bool yaEsta = false;
151         for (auto itL = labsUnicos.iterator(); !itL.isEnd(); itL.next())
152             if (itL.data() == lab) { yaEsta = true; break; }
153         }
154         if (!yaEsta) labsUnicos.insertAtEnd(lab);
155     }
156     std::cout << "\nLaboratorios que suministran 'ACEITES': " << labsUni
157     int idx = 1;
158     for (auto itL = labsUnicos.iterator(); !itL.isEnd(); itL.next()) {
159         std::cout << idx << ". " << *itL.data() << "\n";
160         ++idx;
161     }
162     if (labsUnicos.size() == 0) std::cout << "(Ninguno)\n";
163 }
```

Caso 3

Captura “Antes”

```
122
123 void MediExpress::suministrarMed(const PaMedicamento& med, Laboratorio&
124     bool labExists = false;
125
126     for (auto it = m_lab.iterator(); !it.isEnd(); it.next()) {
127         if (it.data() == lab) {
128             labExists = true;
129             break;
130         }
131     }
132     if (labExists){
133         //Buscar el medicamento y enlazar el primero que coincida
134         const unsigned int n = m_med.len();
135         for (unsigned int i = 0; i < n; ++i) {
136             if (m_med[i] == med) {
137                 m_med[i].setServidoPor(&lab);
138                 break;
139         }
140         }
141     }
142 }
```

Promt

Actúa como un experto en C++ y Google Test. Estoy realizando un análisis de cobertura de caja blanca. Tengo una función en la que una rama específica NO se está ejecutando con mis tests actuales.

CONTEXTO DE MIS TESTS: Te adjunto mi archivo de pruebas actual (test_main.cpp) para que conozcas la estructura y los Fixtures que utilizo.

CÓDIGO DE LA FUNCIÓN COMPLETA:

```
void MediExpress::suministrarMed(const PaMedicamento& med, Laboratorio& lab) const {
    bool labExists = false;
    for (auto it = m_lab.iterator(); !it.isEnd(); it.next()) {
        if (it.data() == lab) {
            labExists = true;
            break;
        }
    }
    if (labExists) {
        const unsigned int n = m_med.len();
        for (unsigned int i = 0; i < n; ++i) {
```

```
            if (m_med[i] == med) {
                m_med[i].setServidoPor(&lab);
                break;
            }
        }
    }
}
```

BLOQUE NO CUBIERTO (marcado en rojo): La función completa nunca se llama desde los tests actuales. El objetivo concreto es cubrir la rama if (labExists) cuando es false: el laboratorio pasado como argumento NO existe en el sistema, por lo que el bloque de asignación no debe ejecutarse.

REQUISITOS OBLIGATORIOS:
Genera ÚNICAMENTE un bloque TEST_F que llame a suministrarMed() directamente sobre un objeto MediExpress instanciado en el test, pasándole un Laboratorio externo con id=99 que no esté registrado en el sistema.
Para obtener un medicamento sin lab, crea 3 medicamentos y 1 laboratorio: autoLinkMedications() asignará lab a los 2 primeros y el 3.º quedará sin lab. Usa getMedicamSinLab() para obtener su referencia.
Verifica que tras llamar a suministrarMed(), el medicamento sigue sin laboratorio asignado (getMedicamSinLab().len() == 1).
Usa el Fixture SimulationTest (que ya gestiona la creación y limpieza de ficheros temporales). NO generes #include, ni main(), ni reescribas el archivo.

Test

```
TEST_F(SimulationTest, SuministrarMed_NoOp_WhenLabNotInSystem) {
    writeFile(TEST_MEDS,
        "1;MED001;Paracetamol;\n"
        "2;MED002;Ibuprofeno;\n"
        "3;MED003;Omeprazol;\n");
    writeFile(TEST_LABS,
        "1;FarmaGen S.L.;Calle Mayor 1;18001;Granada\n");

    MediExpress me(TEST_MEDS, TEST_LABS);

    VDinamico<PaMedicamento*> sinLab = me.getMedicamSinLab();
    ASSERT_EQ(sinLab.len(), 1u) << "Omeprazol (med 3) debe quedar sin lab tras
autoLink.";

    Laboratorio externalLab(99, "Lab Externo", "Calle Z", "00000", "Desconocida");
    me.suministrarMed(*sinLab[0], externalLab);

    VDinamico<PaMedicamento*> sinLabDespues = me.getMedicamSinLab();
    EXPECT_EQ(sinLabDespues.len(), 1u)
        << "suministrarMed no debe asignar nada si el lab no esta en el sistema.";
}
```

Captura “Después”

```
void MediExpress::suministrarMed(const PaMedicamento& med, Laboratorio&
    bool labExists = false;

    for (auto it = m_lab.iterator(); !it.isEnd(); it.next()) {
        if (it.data() == lab) {
            labExists = true;
            break;
        }
    }
    if (labExists){
        //Buscar el medicamento y enlazar el primero que coincida
        const unsigned int n = m_med.len();
        for (unsigned int i = 0; i < n; ++i) {
            if (m_med[i] == med) {
                m_med[i].setServidoPor(&lab);
                break;
        }
        }
    }
}
```

Evolución de la cobertura

Fichero .cpp	Inicial (3 test)	Caso 1: safe_read	Caso 2: aceites	Caso 3: suministrarMed
core_logic.cpp	92% (131/142)	96% (136/142)	96% (137/142)	96% (137/142)
MediExpress.cpp	80% (88/110)	80% (88/110)	91% (100/110)	91% (100/110)
Laboratorio.cpp	43%	43%	43%	43%
PaMedicamento.cpp	63%	63%	63%	63%
utils.cpp	83%	83%	83%	83%

main.cpp | MediExpress.cpp | core_logic.cpp M | CMakeLists.txt M

src > core_logic.cpp > prueba_lista_inserciones(ListaEnlazada<int>&)

```cpp
31 static void separador(const std::string &titulo) {
35 }
36
37 bool safe_read(unsigned int& value) {
38     if (std::cin >> value) return true;
39     if (std::cin.eof()) return false;
40     std::cin.clear();
41     std::cin.ignore(10000, '\n');
42     return true;
43 }
44
45 /**
46  * @brief Realiza las inserciones de la Prueba I sobre la lista de enteros.
47  * @param lista Lista enlazada sobre la que operar.
48  */
49 static void prueba_lista_inserciones(ListaEnlazada<int>& lista) {
50     for (int i = 101; i <= 200; ++i) lista.insertAtEnd(i);
51     separador("Insertados valores 101 a 200 al final");
52     mostrarLista(lista);
53     for (int i = 98; i >= 1; --i) lista.insertAtBiginning(i);
54     separador("Insertados valores 98 a 1 al comienzo");
55     mostrarLista(lista);
56     auto it = lista.iterator();
57     while(!it.isEnd() && it.data() != 101) it.next();
58     if(!it.isEnd()) lista.InsertBefore(it, 100);
59     separador("Insertado 100 delante de 101");
60     mostrarLista(lista);
61     it = lista.iterator();
62     while(!it.isEnd() && it.data() != 98) it.next();
63     if(!it.isEnd()) lista.insertAfter(it, 99);
```

PROBLEMS | OUTPUT | TEST RESULTS | TERMINAL | SONARQUBE | DEBUG CONSOLE | PORTS

```
PS C:\GitHub\calidad_software_practicas\P4B9> py -m gcovr -r . --exclude-throw-branches --exclude-
>>
(INFO) Reading coverage data...
(INFO) Writing coverage report...
PS C:\GitHub\calidad_software_practicas\P4B9>
```

main* | 0 0 | Ejecutar Programa (P4B9) | Build | SonarQube | File 96% Covered | Recon

main.cpp | MediExpress.cpp | core_logic.cpp M | CMakeLists.txt M

src > MediExpress.cpp > ...

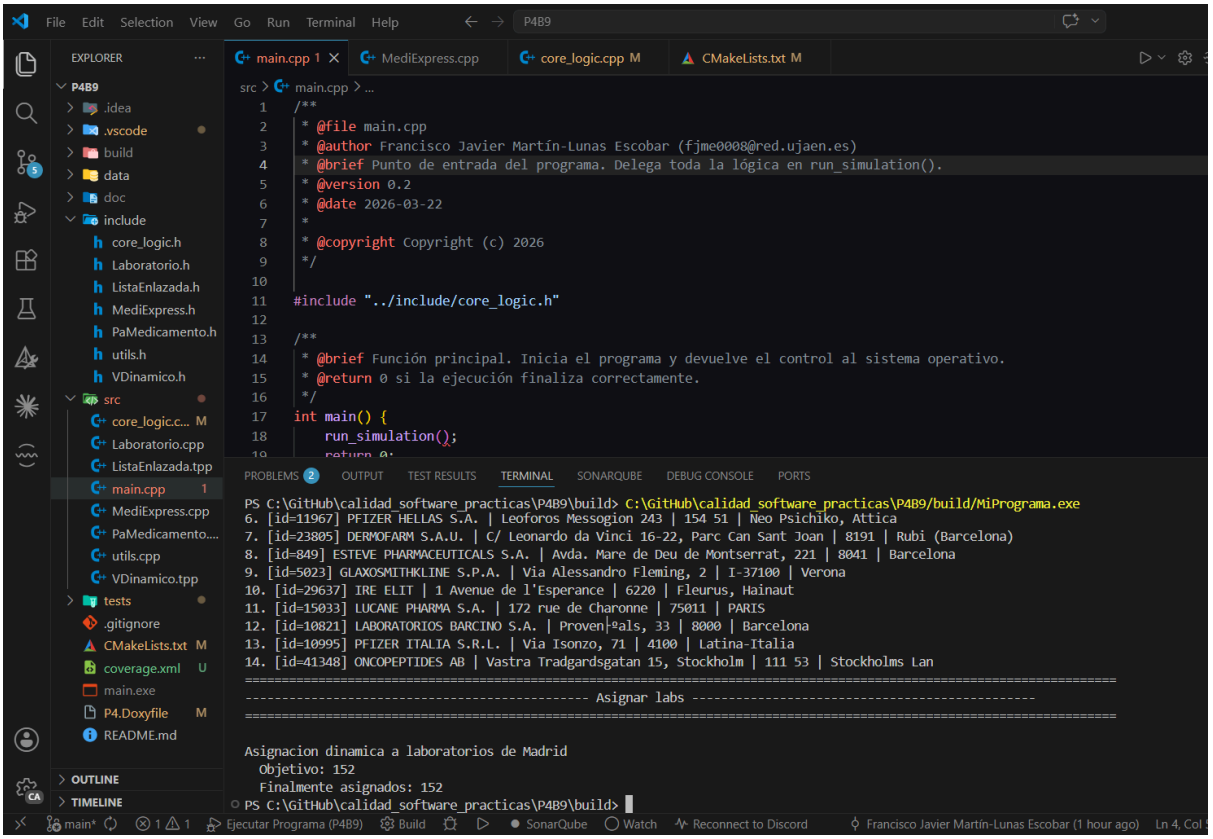
```cpp
MediExpress::MediExpress(const std::string &csvPathVD, const std::string &csvPathLE){
    autoLinkMedications();
}

MediExpress::~MediExpress() = default;

/**
 * Metodos
 */

void MediExpress::suministrarMed(const PaMedicamento& med, Laboratorio& lab) const{
    bool labExists = false;

    for (auto it = m_lab.iterator(); !it.isEnd(); it.next()) {
        if (it.data() == lab) {
            labExists = true;
            break;
        }
    }
    if (labExists){
        //Buscar el medicamento y enlazar el primero que coincida
        const unsigned int n = m_med.len();
        for (unsigned int i = 0; i < n; ++i) {
            if (m_med[i] == med) {
                m_med[i].setServidoPor(&lab);
                break;
        }
        }
    }
}
```

PROBLEMS | OUTPUT | TEST RESULTS | TERMINAL | SONARQUBE | DEBUG CONSOLE | PORTS

```
PS C:\GitHub\calidad_software_practicas\P4B9> py -m gcovr -r . --exclude-throw-branches --exclude-unreachab
>>
(INFO) Reading coverage data...
(INFO) Writing coverage report...
PS C:\GitHub\calidad_software_practicas\P4B9>
```

main* | 0 0 | Ejecutar Programa (P4B9) | Build | SonarQube | File 90% Covered | Reconnect to Disco

Lanzamiento a producción



s